ARTIFICIAL INTELLIGENCE INTERNSHIP PROJECT REPORT

HANDWRITTEN DIGIT RECOGNITION USING NEURAL NETWORKS

A project report submitted in fulfilment of the requirements of the Artificial Intelligence Internship Programme (September 2026 batch)

Submitted by

SANGINI AGRAWAL

NAVIOTECH SOLUTION

Artificial Intelligence Domain

October 2026

CERTIFICATE

This is to certify that the project report entitled **"Handwritten Digit Recognition using Neural Networks"** submitted by **Sangini Agrawal** is a record of bona fide work carried out under the Artificial Intelligence Internship Programme (September 2026 batch) of Naviotech Solution.

The work presented in this report has been carried out in accordance with the project guidelines issued by the organisation.

DECLARATION

I, **Sangini Agrawal**, hereby declare that the project report entitled **"Handwritten Digit Recognition using Neural Networks"** submitted to Naviotech Solution as part of the Artificial Intelligence Internship Programme is a record of my own work. The dataset, software libraries and published works used in this project have been duly acknowledged and cited in the report.

Sangini Agrawal
AI Intern, September 2026 batch

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Naviotech Solution** for giving me the opportunity to carry out this project as part of the Artificial Intelligence Internship Programme. I am thankful to the Academic Head and the academic team for the project allocation session, the project guide manual and the starter material, which provided a clear structure for the work.

I thank my project guide and mentors for their guidance on project development, documentation and submission, and the support team for promptly answering questions during the internship.

I also acknowledge Yann LeCun, Corinna Cortes and Christopher J. C. Burges for creating the MNIST database of handwritten digits and making it freely available, the U.S. National Institute of Standards and Technology (NIST), whose handwriting collections it is built from, and the developers of TensorFlow, Keras, NumPy, scikit-learn and Pillow, whose open-source tools made this work possible.

Finally, I am grateful to my family and friends for their constant encouragement and support.

Sangini Agrawal

ABSTRACT

Handwritten digits appear on cheques, postal envelopes, examination sheets and paper forms, and reading them automatically is a classic problem in pattern recognition. This project builds and evaluates neural networks that recognise handwritten digits (0–9) from 28 × 28 grayscale images of the MNIST dataset, which contains 70,000 labelled samples. The pixel values were scaled to the range 0–1 and 6,000 of the 60,000 training images were held out as a stratified validation set, so that the 10,000 test images were used only once, for the final evaluation.

Two models were implemented in TensorFlow/Keras. The main model, a dense (fully connected) neural network with two hidden layers of 128 and 64 ReLU neurons, 20% dropout and a softmax output (109,386 parameters), reached a test accuracy of 97.93%. An improved convolutional neural network with two convolution–pooling blocks (34,826 parameters) reached 99.19% with a macro F1-score of 0.9918, making 81 errors on 10,000 images. Both exceed the project target of 95% and outperform logistic regression (92.45%), k-nearest neighbours (96.99%) and a random forest (96.96%). Error analysis shows that most mistakes involve digits with similar shapes, such as 4 and 9 or 7 and 2, and most of them are made with low confidence.

A prediction system lets users test their own handwriting by drawing on a canvas in Google Colab, uploading a photo, using a command-line tool or a browser demo. User images are prepared the same way as MNIST digits (inverted, cropped, fitted into a 20 × 20 box and centred). With this preprocessing the CNN read all eleven sample images correctly. On 30 harder photo-style images it got 16 right, compared with only 4 with a plain resize, which shows both why preprocessing matters and that small digits on noisy paper are still difficult.

Keywords: *handwritten digit recognition, MNIST, neural networks, convolutional neural networks, deep learning, TensorFlow, Keras, image classification, OCR*

TABLE OF CONTENTS

CERTIFICATE.....i
DECLARATION.....ii
ACKNOWLEDGEMENT.....iii
ABSTRACT.....iv
LIST OF FIGURES.....vii
LIST OF TABLES.....viii
LIST OF ABBREVIATIONS.....ix
CHAPTER 1 INTRODUCTION.....1
1.1 Background.....1
1.2 Neural Networks for Image Recognition.....1
1.3 Problem Statement.....1
1.4 Objectives.....1
1.5 Scope of the Project.....2
1.6 Applications.....2
1.7 Organisation of the Report.....2
CHAPTER 2 LITERATURE REVIEW.....3
2.1 The MNIST Database.....3
2.2 Classical Approaches.....3
2.3 Fully Connected Neural Networks.....3
2.4 Convolutional Neural Networks.....3
2.5 Summary of Related Work.....4
2.6 Research Gap and Contribution.....4
CHAPTER 3 METHODOLOGY.....5
3.1 Overall Workflow.....5
3.2 Dataset Description.....5
3.3 Exploratory Data Analysis.....5
3.4 Data Preprocessing.....7
3.5 Neural Network Fundamentals.....8
3.6 Convolutional Neural Networks.....9
3.7 Evaluation Metrics.....9
3.8 Baseline Models.....10
CHAPTER 4 SYSTEM DESIGN.....11
4.1 System Architecture.....11
4.2 Model 1: Dense Neural Network.....11
4.3 Model 2: Convolutional Neural Network.....12
4.4 Hyper-parameters.....12
4.5 User Input Preprocessing.....13
4.6 Software and Hardware Requirements.....13
4.7 Project Structure.....14

CHAPTER 5 IMPLEMENTATION **15**
5.1 Environment Setup 15
5.2 Loading the Data 15
5.3 Preprocessing 15
5.4 Building the Dense Network 15
5.5 Training 16
5.6 Building and Training the CNN 17
5.7 Evaluation 17
5.8 Prediction System 18
5.9 Browser Demo 18
5.10 Saving the Models 19
CHAPTER 6 RESULTS AND DISCUSSION **20**
6.1 Training Behaviour 20
6.2 Performance on the Test Set 20
6.3 Per-Digit Results 21
6.4 Confusion Matrices 22
6.5 Error and Confidence Analysis 24
6.6 Comparison with Classical Machine Learning 25
6.7 Comparison with Published Work 26
6.8 Recognising Users' Handwriting 26
6.9 Discussion 27
6.10 Limitations 27
CHAPTER 7 CONCLUSION **28**
CHAPTER 8 FUTURE SCOPE **29**
REFERENCES **30**
APPENDIX A SOURCE CODE **32**

LIST OF FIGURES

Figure 3.1: The first 30 training images and their labels....6
Figure 3.2: Number of images per digit in the training and test sets....6
Figure 3.3: The average image of each digit over all training images....6
Figure 3.4: Distribution of pixel values (logarithmic scale)....7
Figure 4.1: System architecture....11
Figure 4.2: Architecture of the dense neural network....11
Figure 4.3: Architecture of the convolutional neural network....12
Figure 4.4: Preprocessing of a photo-style user digit....13
Figure 5.1: Output: dataset loaded....15
Figure 5.2: Output: dense network summary....16
Figure 5.3: Output: training log of the dense network....16
Figure 5.4: Output: training log of the CNN....17
Figure 5.5: Output: test accuracy and classification report....17
Figure 5.6: Output: the CNN reading an 8 drawn on the Colab canvas....18
Figure 5.7: Browser demo: the CNN reading a hand-drawn 4....19
Figure 6.1: Training and validation accuracy and loss for both networks....20
Figure 6.2: Accuracy for each digit....21
Figure 6.3: Confusion matrix of the dense network....22
Figure 6.4: Confusion matrix of the CNN....23
Figure 6.5: CNN predictions on the first 24 test images....24
Figure 6.6: Examples of the CNN's mistakes (prediction, confidence and true label)....24
Figure 6.7: Confidence of correct and wrong CNN predictions....25
Figure 6.8: Test accuracy of neural networks and classical baselines....25

LIST OF TABLES

Table 2.1: Published MNIST results (test error)..... 4
Table 3.1: Project phases..... 5
Table 3.2: Dataset summary..... 5
Table 3.3: Images per digit..... 6
Table 3.4: Data split..... 7
Table 3.5: Evaluation metrics..... 9
Table 4.1: Dense network layers and parameters..... 11
Table 4.2: CNN layers and parameters..... 12
Table 4.3: Training hyper-parameters..... 12
Table 4.4: Preprocessing steps for user images..... 13
Table 4.5: Software requirements..... 13
Table 4.6: Hardware requirements..... 14
Table 4.7: Modules and responsibilities..... 14
Table 6.1: Test-set performance (10,000 images)..... 20
Table 6.2: Per-digit recall (accuracy) and F1-score..... 21
Table 6.3: Test accuracy of all models..... 25
Table 6.4: Accuracy on photo-style user images..... 26

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
NN	Neural Network
MLP	Multilayer Perceptron (dense neural network)
CNN	Convolutional Neural Network
MNIST	Modified National Institute of Standards and Technology (database)
NIST	National Institute of Standards and Technology
OCR	Optical Character Recognition
ReLU	Rectified Linear Unit
Adam	Adaptive Moment Estimation (optimiser)
SGD	Stochastic Gradient Descent
kNN	k-Nearest Neighbours
RGB	Red, Green, Blue (colour image)
F1	Harmonic mean of precision and recall
GPU	Graphics Processing Unit

CHAPTER 1
INTRODUCTION

1.1 Background

Even today, a lot of information is still written by hand: amounts on bank cheques, postal codes on envelopes, marks on examination sheets and entries on paper forms. Converting this handwriting into digital data by hand is slow, expensive and error-prone, so automatic handwriting recognition has been an active research area for decades [4].

Recognising even a single handwritten digit is harder than it looks. Every person writes differently: digits vary in slant, size, stroke thickness and style, a 4 may be open or closed, a 7 may have a crossbar, and a 1 may be a single stroke or have a base. A rule-based program that tries to describe each digit by hand cannot cover this variety. Machine learning solves the problem differently: the computer is shown many labelled examples and learns the patterns that distinguish the digits by itself.

1.2 Neural Networks for Image Recognition

An image is a grid of numbers. A grayscale image of 28 × 28 pixels is simply 784 numbers between 0 (black) and 255 (white). A neural network takes these numbers as input and, through layers of simple computing units, learns to map them to the correct class. **Convolutional neural networks (CNNs)**, inspired by the visual cortex [5], [6], go further: they scan the image with small learned filters to detect local features such as edges, curves and loops wherever they appear. CNNs have driven the success of deep learning in computer vision [9], [16], and one of their first real uses was reading handwritten digits on bank cheques [1].

1.3 Problem Statement

Given a 28 × 28 grayscale image of a single handwritten digit, build a system that automatically identifies which digit (0–9) it shows with an accuracy of more than 95%, evaluate it properly on unseen images, and allow users to test it with their own handwriting.

1.4 Objectives

The objectives set out in the Naviotech project guide are to:

1. Understand the fundamentals of neural networks.
2. Learn image classification techniques.
3. Train a model using the MNIST dataset.
4. Evaluate model accuracy and performance.

5. Deploy a simple digit recognition system.

The expected outcome is a digit recognition accuracy above 95%, practical understanding of image classification and experience with deep learning frameworks.

1.5 Scope of the Project

The project deals only with single, isolated handwritten digits. The work includes data exploration, preprocessing, two neural network architectures (a dense network as the main model and a CNN as an improved model), training with early stopping, evaluation with standard classification metrics, comparison with classical machine-learning algorithms, and a prediction system that accepts drawn or photographed digits. Recognition of multi-digit numbers, letters or full handwritten text is outside the scope.

1.6 Applications

- **Bank cheque processing:** reading the amount written in digits on cheques, one of the first commercial uses of CNNs [1].
- **Postal code recognition:** sorting mail automatically by reading handwritten postcodes and PIN codes.
- **OCR systems:** digit recognition is a building block of optical character recognition for scanned documents.
- **Educational tools:** automatic marking of numeric answers and interactive learning applications.
- **Automated form processing:** extracting phone numbers, dates and amounts from paper forms, surveys and invoices.

1.7 Organisation of the Report

Chapter 2 reviews related work on MNIST and neural networks. Chapter 3 describes the dataset, the exploratory analysis, the preprocessing and the theory behind dense and convolutional networks and the evaluation metrics. Chapter 4 presents the system design and both architectures. Chapter 5 explains the implementation with code excerpts and program outputs. Chapter 6 reports and discusses the results. Chapters 7 and 8 give the conclusion and future scope, followed by the references and an appendix with source code.

CHAPTER 2
LITERATURE REVIEW

2.1 The MNIST Database

MNIST was created by LeCun, Cortes and Burges [1], [2] from two handwriting collections of the U.S. National Institute of Standards and Technology: one written by Census Bureau employees and one by high-school students. Each set of MNIST mixes both groups, and the writers of the training images are different from the writers of the test images, so the test set measures how well a model generalises to new people's handwriting.

The original black-and-white digits were size-normalised to fit in a 20 × 20 pixel box while keeping their aspect ratio. Anti-aliasing during resizing introduced grey levels. The digits were then centred in a 28 × 28 image by their centre of mass. This preprocessing is important for this project: user images must be prepared in the same way to be recognised reliably (Section 4.5). MNIST has become the standard first benchmark for image classification [3].

2.2 Classical Approaches

LeCun et al. [1] compared many methods on MNIST. A linear classifier (equivalent to a one-layer network) had a test error of 12.0%, and a k-nearest-neighbour classifier with Euclidean distance had 5.0% error. k-nearest neighbours [18] stores all training images and labels a new image by a vote among the most similar ones; it is simple but slow at prediction time and sensitive to small shifts. Random forests [19], ensembles of decision trees, are another strong general-purpose classifier and are used as a baseline in this project.

2.3 Fully Connected Neural Networks

Multilayer perceptrons trained with back-propagation [10] were among the first neural networks applied to digits. LeCun et al. [1] reported 4.7% error for a network with one hidden layer of 300 units and 2.95% for two hidden layers of 500 and 150 units. Later techniques made such networks much better: the ReLU activation [11], careful weight initialisation [14], dropout regularisation [12], adaptive optimisers such as Adam [13] and early stopping [15]. With elastic distortions of the training images, Simard et al. [7] reduced the error of a two-layer network to 0.7%.

2.4 Convolutional Neural Networks

Hubel and Wiesel [5] found that neurons in the visual cortex respond to simple patterns in small regions of the visual field. Fukushima's Neocognitron [6] translated this idea into a hierarchical network that was tolerant to shifts in position. LeCun et al. [1] combined

convolution, sub-sampling and gradient-based learning in LeNet-5, which achieved 0.95% error on MNIST and was deployed commercially to read cheques. Simard et al. [7] reached 0.4% error with a CNN trained on distorted images, and Cireşan et al. [8] reached 0.23% with a committee of deep CNNs, close to human performance. The success of deep CNNs on large image datasets [9] made them the standard approach in computer vision [16], [17].

2.5 Summary of Related Work

Table 2.1: Published MNIST results (test error)

Study	Method	Test error
LeCun et al., 1998 [1]	Linear classifier (1-layer network)	12.0%
LeCun et al., 1998 [1]	k-nearest neighbours, Euclidean distance	5.0%
LeCun et al., 1998 [1]	2-layer network, 300 hidden units	4.7%
LeCun et al., 1998 [1]	3-layer network, 500 + 150 hidden units	2.95%
LeCun et al., 1998 [1]	LeNet-5 convolutional network	0.95%
Simard et al., 2003 [7]	CNN with elastic distortions	0.4%
Cireşan et al., 2012 [8]	Committee of 35 deep CNNs	0.23%
This project	Dense network 784-128-64-10	2.07%
This project	CNN, two conv–pool blocks	0.81%

2.6 Research Gap and Contribution

Many MNIST tutorials, including the starter material for this project, stop at the test accuracy. They often use the test set for validation, use a sigmoid output with a categorical loss, and feed user images into the model with a simple resize, which does not work well on real photos. This project tries to improve on this. A separate validation set is kept so that the test score is fair, a dense network and a CNN are compared in the same way along with three classical models, the mistakes and the model's confidence are studied closely, and user images are prepared like MNIST digits so that the model can read real handwriting in Colab, on the command line and in a web browser.

CHAPTER 3
METHODOLOGY

3.1 Overall Workflow

The work follows the six phases of the Naviotech project guide, extended with an improved model and a comparison with classical methods (Table 3.1).

Table 3.1: Project phases

Phase	Activity	Output
1. Data collection	Load MNIST, explore shapes, samples, class balance	Arrays of images and labels
2. Data preprocessing	Normalise pixels, validation split, reshape	Model-ready arrays
3. Model development	Design the dense network (and the CNN)	Compiled Keras models
4. Model training	Adam optimiser, early stopping, monitor accuracy and loss	Trained weights, learning curves
5. Testing and evaluation	Metrics on 10,000 unseen images, error analysis, baselines	Accuracy, F1, confusion matrices
6. Prediction	Users draw or upload digits; preprocessing; prediction	Colab canvas, CLI, browser demo

3.2 Dataset Description

Table 3.2: Dataset summary

Property	Value
Name	MNIST database of handwritten digits [2]
Total images	70,000 (60,000 training, 10,000 test)
Image size	28 × 28 pixels, single channel (grayscale)
Pixel values	Integers 0 (black) to 255 (white); digit white on black
Classes	10 (digits 0–9)
Access	keras.datasets.mnist.load_data(), downloaded automatically

3.3 Exploratory Data Analysis

Figure 3.1 shows the first 30 training images. Even in these few images the handwriting varies a lot: some 1s are slanted strokes and some vertical, 4s are open or closed, and 7s may carry a crossbar.

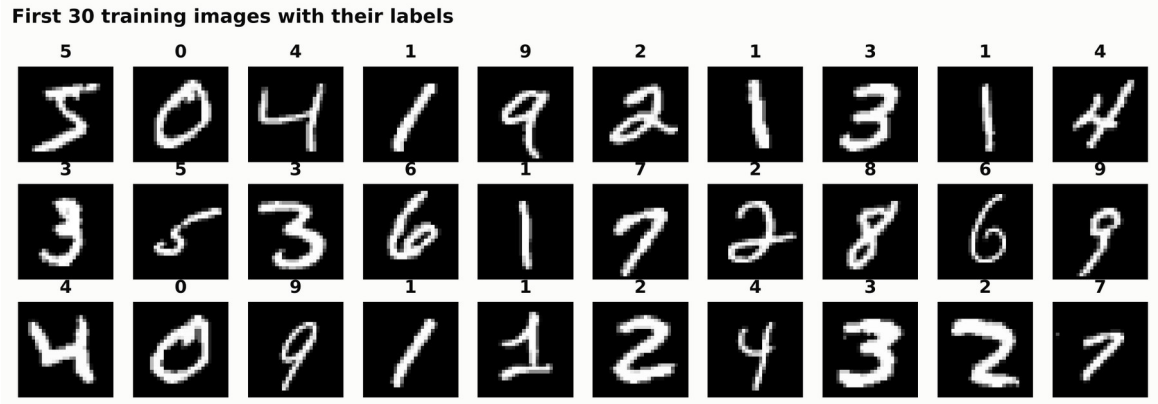


Figure 3.1: The first 30 training images and their labels

The classes are roughly balanced (Figure 3.2). In the training set the most frequent digit is 1 (6,742 images) and the least frequent is 5 (5,421). In the test set every digit has between 892 and 1,135 images. Because no digit dominates, accuracy is a fair summary measure, complemented by per-class metrics.

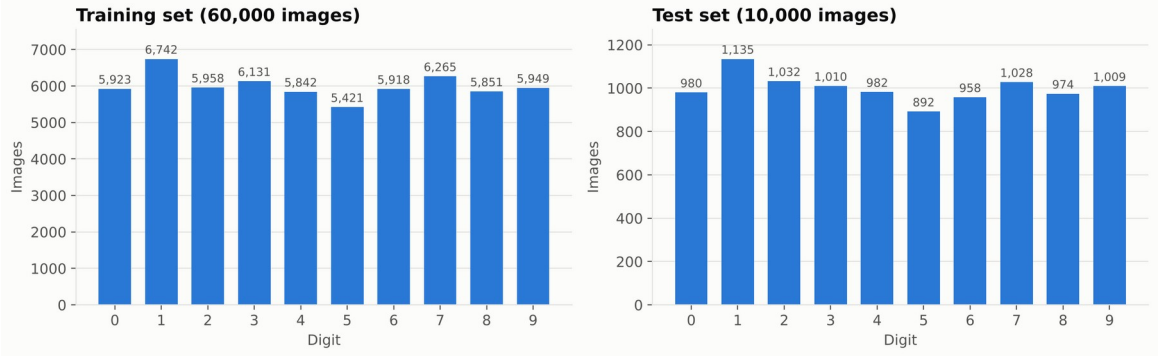


Figure 3.2: Number of images per digit in the training and test sets

Table 3.3: Images per digit

Digit	0	1	2	3	4	5	6	7	8	9
Training	5,923	6,742	5,958	6,131	5,842	5,421	5,918	6,265	5,851	5,949
Test	980	1,135	1,032	1,010	982	892	958	1,028	974	1,009

Averaging all training images of each digit (Figure 3.3) gives a clear, blurred prototype for every class. The blur shows where writers differ; the bright core shows the strokes almost everyone shares.



Figure 3.3: The average image of each digit over all training images

Figure 3.4 shows the distribution of pixel values. About 81% of pixels are pure black background, and most of the rest are either fully white stroke pixels or grey anti-aliased edges. The information that distinguishes the digits is concentrated in a small central area.

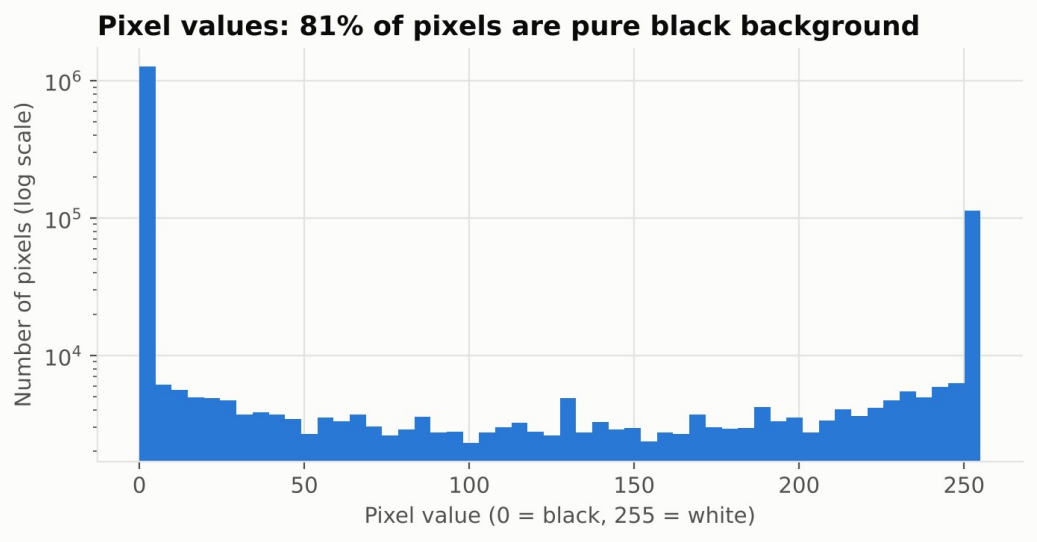


Figure 3.4: Distribution of pixel values (logarithmic scale)

3.4 Data Preprocessing

3.4.1 Normalisation

Pixel values were divided by 255 to map them from 0–255 to 0–1:

$$x' = x \,/\, 255$$

When all inputs are small and on the same scale, the sums inside the network stay in a sensible range, so training is faster and more stable.

3.4.2 Validation set

The step-by-step guide that came with the starter material checks the model on the test images during training. That lets information from the test set influence choices such as when to stop training, which makes the final test score optimistic. To avoid this, 6,000 of the 60,000 training images were randomly taken, stratified by digit, as a validation set (Table 3.4). The 10,000 test images were used only once, after training was complete.

Table 3.4: Data split

Subset	Images	Purpose
Training	54,000	Fit the weights
Validation	6,000	Monitor training, early stopping
Test	10,000	Final, unbiased evaluation

3.4.3 Reshaping and labels

The dense network receives each image as a 28 × 28 array and flattens it into a vector of 784 values in its first layer. The CNN needs an explicit channel dimension, so images are reshaped from 28 × 28 to 28 × 28 × 1. Labels are kept as integers 0–9 and used with the sparse categorical cross-entropy loss, which avoids converting them to one-hot vectors.

3.5 Neural Network Fundamentals

3.5.1 Neurons and dense layers

Each neuron computes a weighted sum of its inputs plus a bias and applies an activation function f:

$$a = f(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b)$$

In a dense layer every neuron is connected to every input. A layer with n inputs and m neurons has n × m weights and m biases, which is why the first layer of the dense network (784 inputs, 128 neurons) contains most of its parameters.

3.5.2 Activation functions

Hidden layers use the rectified linear unit, ReLU(z) = max(0, z) [11]. It is cheap to compute, does not saturate for positive values and lets stacked layers represent non-linear decision boundaries. The output layer uses softmax, which turns the ten output scores into probabilities that are positive and sum to one:

$$softmax(z_i) = e^{(z_i)} / \Sigma_j\, e^{(z_j)},\ \ j = 0 \ldots 9$$

The starter code used a sigmoid output, which treats each of the ten outputs independently; its outputs need not sum to one and are not a proper probability distribution over mutually exclusive digits. Softmax is the correct pairing with categorical cross-entropy.

3.5.3 Loss function and optimiser

Training minimises the sparse categorical cross-entropy, the average negative log-probability of the correct digit:

$$L = -(1/N)\ \Sigma\ log\ p(y_i \mid x_i)$$

The weights are updated by back-propagation [10] with the Adam optimiser [13], which adapts the step size of each weight using running averages of its gradient and squared gradient. A learning rate of 0.001 and mini-batches of 128 images were used, which gives 422 weight updates per epoch.

3.5.4 Regularisation

Dropout [12] randomly sets a fraction of a layer's outputs to zero at every training step (20% in the dense network, 50% before the CNN's output layer), preventing neurons from co-

adapting and reducing over-fitting. Early stopping [15] monitors the validation loss and stops training when it has not improved for a set number of epochs (5 for the dense network, 3 for the CNN), restoring the best weights.

3.6 Convolutional Neural Networks

3.6.1 Convolution

A convolutional layer slides small filters (here 3 × 3) across the image. At each position it computes a weighted sum of the 9 pixels under the filter, producing a feature map that is high where the filter's pattern is present. A layer with 32 filters produces 32 feature maps, each detecting a different pattern such as a horizontal edge, a diagonal stroke or a curve end. With no padding and stride 1, a 3 × 3 filter reduces each side by 2 pixels:

$$\textit{output size} = \textit{input size} - \textit{kernel size} + 1 \quad (28 - 3 + 1 = 26)$$

3.6.2 Pooling

A 2 × 2 max-pooling layer keeps the largest value in each 2 × 2 block, halving the width and height. This reduces computation and makes the network tolerant to small shifts of the digit.

3.6.3 Why CNNs suit images

- **Local connectivity:** each output looks only at a small neighbourhood, matching the local nature of strokes.
- **Weight sharing:** the same filter is used at every position, so a pattern is recognised wherever it appears and far fewer parameters are needed.
- **Hierarchy:** early layers find edges, later layers combine them into loops and corners, and the classifier combines those into digits.

A dense network, by contrast, treats the 784 pixels as an unordered list and has to learn separately that a stroke shifted by one pixel is still the same stroke.

3.7 Evaluation Metrics

With ten classes, precision, recall and F1-score are computed for each digit and then averaged with equal weight for every class (macro average).

Table 3.5: Evaluation metrics

Metric	Definition (per digit d)	Meaning
Accuracy	correct predictions / all predictions	Overall share of digits read correctly
Precision	TP / (TP + FP)	When the model says d, how often is it really d?
Recall	TP / (TP + FN)	Of all real d's, how many were found? (per-class accuracy)
F1-score	2 · P · R / (P + R)	Balance of precision and recall
Confusion matrix	10 × 10 table of true vs predicted digit	Which digits are mistaken for which

3.8 Baseline Models

To check whether neural networks were really needed, three classical algorithms were also trained on the same 54,000 flattened training images and tested them on the same test set: multinomial logistic regression, k-nearest neighbours with k = 3, and a random forest of 100 trees.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

Figure 4.1 shows the system. Training data flow from the MNIST dataset through normalisation and splitting into the two networks, which are evaluated and saved. At prediction time, a user's image passes through MNIST-style preprocessing before the saved model classifies it.

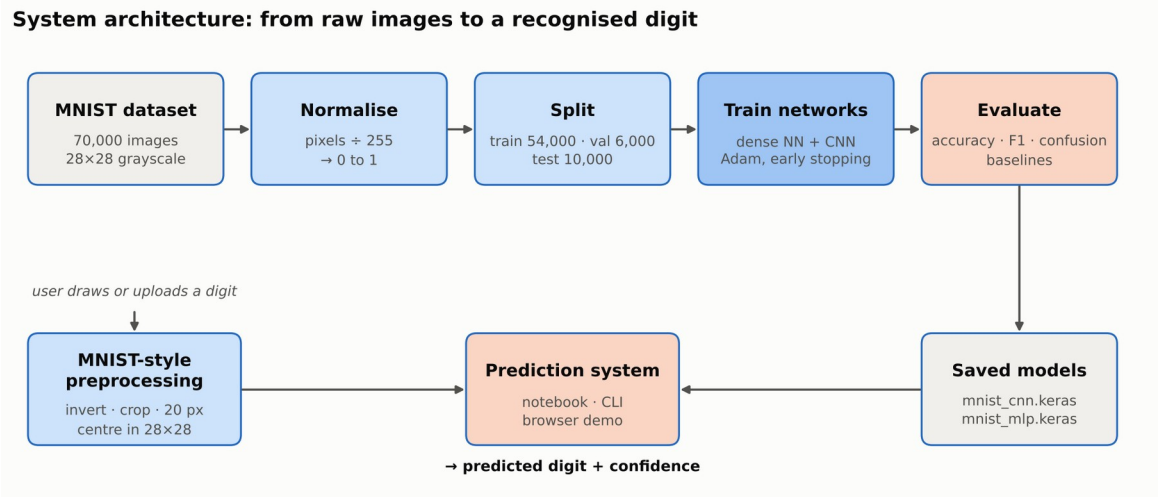


Figure 4.1: System architecture

4.2 Model 1: Dense Neural Network

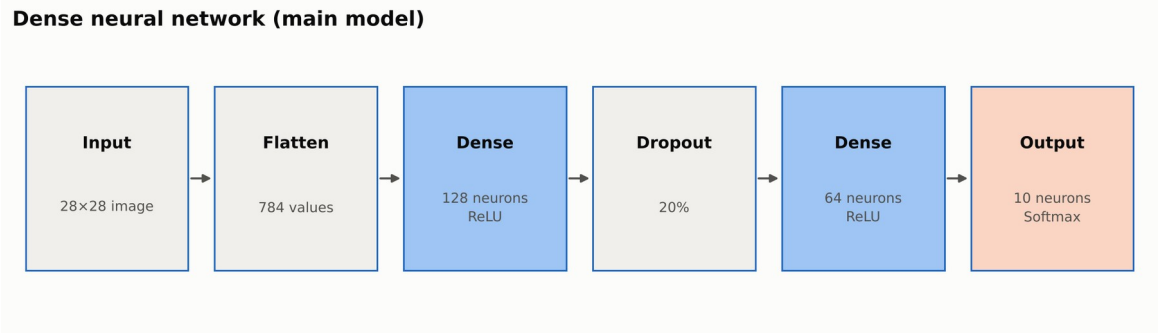


Figure 4.2: Architecture of the dense neural network

Table 4.1: Dense network layers and parameters

Layer	Output shape	Activation	Parameters
Input	(28, 28)	–	0
Flatten	(784)	–	0
Dense (hidden_1)	(128)	ReLU	784 × 128 + 128 = 100,480
Dropout 0.2	(128)	–	0
Dense (hidden_2)	(64)	ReLU	128 × 64 + 64 = 8,256

Layer	Output shape	Activation	Parameters
Dense (output)	(10)	Softmax	64 × 10 + 10 = 650
Total			109,386

4.3 Model 2: Convolutional Neural Network

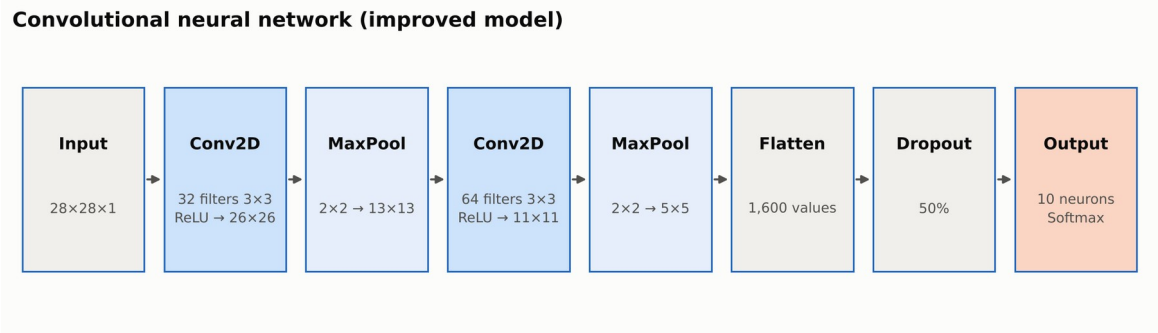


Figure 4.3: Architecture of the convolutional neural network

Table 4.2: CNN layers and parameters

Layer	Output shape	Activation	Parameters
Input	(28, 28, 1)	–	0
Conv2D 32 × (3×3)	(26, 26, 32)	ReLU	3·3·1·32 + 32 = 320
MaxPooling 2×2	(13, 13, 32)	–	0
Conv2D 64 × (3×3)	(11, 11, 64)	ReLU	3·3·32·64 + 64 = 18,496
MaxPooling 2×2	(5, 5, 64)	–	0
Flatten	(1,600)	–	0
Dropout 0.5	(1,600)	–	0
Dense (output)	(10)	Softmax	1,600 × 10 + 10 = 16,010
Total			34,826

Although it is the more powerful model, the CNN has about three times fewer parameters than the dense network (34,826 vs 109,386), because each 3 × 3 filter is shared across the whole image.

4.4 Hyper-parameters

Table 4.3: Training hyper-parameters

Hyper-parameter	Dense network	CNN
Optimiser / learning rate	Adam / 0.001	Adam / 0.001
Loss	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Batch size	128	128
Maximum epochs	30	15
Early stopping patience	5 (val_loss, restore best)	3 (val_loss, restore best)

Hyper-parameter	Dense network	CNN
Dropout	0.2 after hidden layer 1	0.5 before output
Random seed	42	42

4.5 User Input Preprocessing

A digit drawn on a white canvas or photographed on paper differs from MNIST in several ways: it is dark on light, sits on a grey or noisy background, may be small and anywhere in the frame, and is not centred. So the prediction system prepares user images in the same way as MNIST (Section 2.1, Table 4.4, Figure 4.4).

Table 4.4: Preprocessing steps for user images

Step	Operation	Reason
1	Convert to grayscale; invert if the background is light	MNIST digits are white on black
2	Subtract the background level; zero faint pixels relative to the ink strength	Removes grey paper, shadows and noise
3	Crop to the bounding box of clearly inked pixels	Removes empty space around the digit
4	Resize so the longer side is 20 px, keeping the aspect ratio	MNIST digits fit a 20 × 20 box
5	Place in a 28 × 28 frame and shift so the centre of mass is at (14, 14)	MNIST digits are centred by mass
6	Scale to 0–1	Same range as the training data

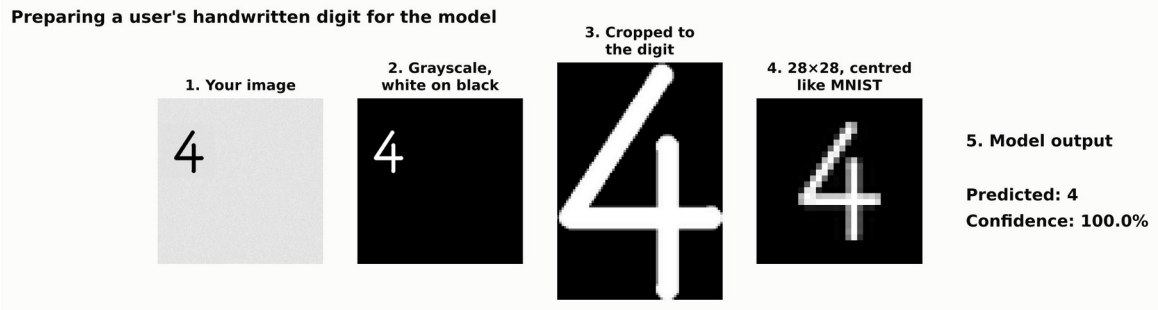


Figure 4.4: Preprocessing of a photo-style user digit

4.6 Software and Hardware Requirements

Table 4.5: Software requirements

Component	Version / tool	Purpose
Language	Python 3.10+	Implementation
Deep learning	TensorFlow 2.16+ with Keras [21], [22]	Build and train the networks
Numerical	NumPy, Pandas	Arrays and tables
Visualisation	Matplotlib	Charts and image grids
Machine learning	scikit-learn 1.3+ [20]	Splitting, metrics, baseline models
Images	Pillow	Reading and preparing user images

Component	Version / tool	Purpose
Environment	Google Colab / Jupyter / VS Code	Development and execution
Web demo	HTML, JavaScript	Browser-based digit recognition
Version control	Git, GitHub	Code hosting and submission

Table 4.6: Hardware requirements

Component	Minimum	Recommended
Processor	64-bit dual-core CPU	Google Colab runtime
Memory	4 GB RAM	8 GB RAM
GPU	Not required	Colab T4 GPU speeds up CNN training
Storage	1 GB free	–

4.7 Project Structure

Table 4.7: Modules and responsibilities

File	Responsibility
src/config.py	Paths, seed, split size and all hyper-parameters
src/data.py	Load MNIST, normalise, stratified validation split
src/model.py	Build and compile the dense network and the CNN
src/train.py	Train, evaluate, benchmark, save models, metrics and charts
src/evaluate.py	Metrics, classification report, baseline models
src/visualize.py	All figures
src/preprocess.py	MNIST-style preparation of user images
src/predict.py	Command-line digit recognition
src/export_web.py	Export CNN weights for the browser demo
web_demo/index.html	Draw-a-digit page running the CNN in JavaScript
notebooks/…ipynb	Complete walkthrough for Google Colab

CHAPTER 5
IMPLEMENTATION

5.1 Environment Setup

All the code was run in Google Colab, where nothing needs to be installed. It can also be run on a computer after pip install -r requirements.txt. The random seeds for Python, NumPy and TensorFlow are fixed so that the results can be repeated.

```
import tensorflow as tf
from tensorflow import keras
SEED = 42
keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()
```

5.2 Loading the Data

```
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.mnist.load_data()
print(x_train_full.shape, x_test.shape)        # (60000, 28, 28) (10000, 28, 28)
```



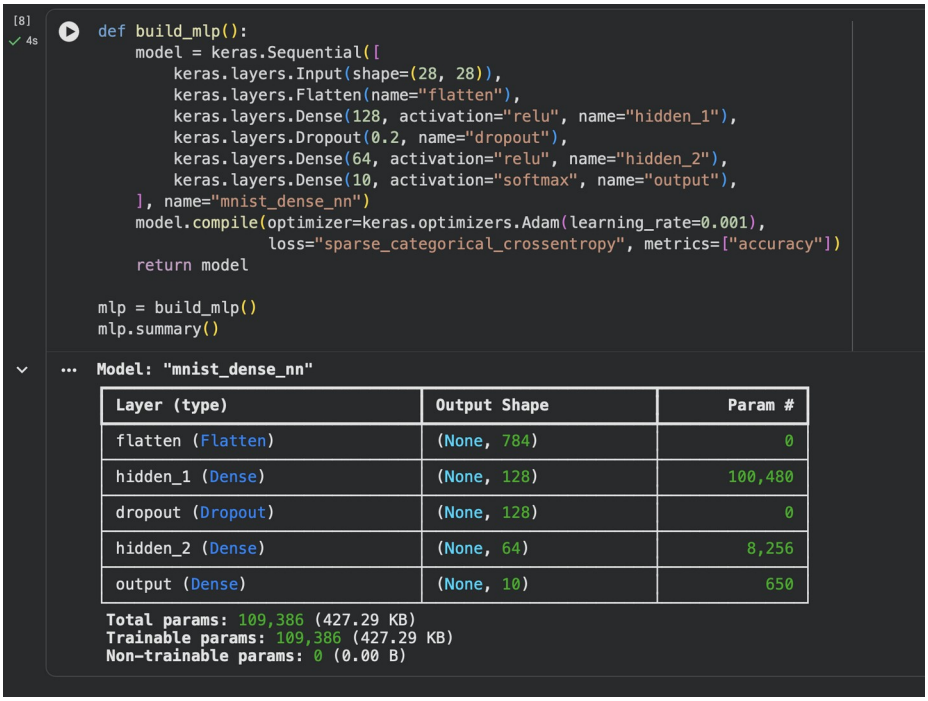
Figure 5.1: Output: dataset loaded

5.3 Preprocessing

```
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full, test_size=6000, stratify=y_train_full,
random_state=SEED)
```

5.4 Building the Dense Network

```
model = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```



```
[8]
✓ 4s
def build_mlp():
    model = keras.Sequential([
        keras.layers.Input(shape=(28, 28)),
        keras.layers.Flatten(name="flatten"),
        keras.layers.Dense(128, activation="relu", name="hidden_1"),
        keras.layers.Dropout(0.2, name="dropout"),
        keras.layers.Dense(64, activation="relu", name="hidden_2"),
        keras.layers.Dense(10, activation="softmax", name="output"),
    ], name="mnist_dense_nn")
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model

mlp = build_mlp()
mlp.summary()
```

Model: "mnist_dense_nn"

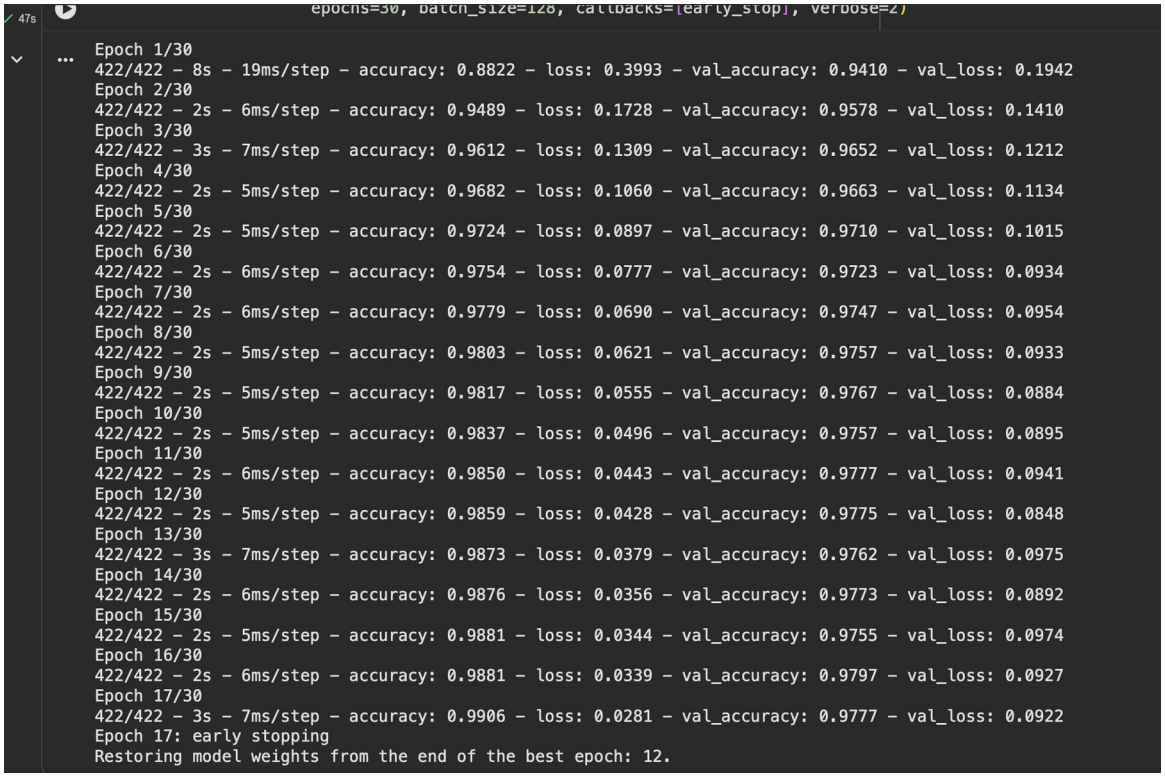
Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
hidden_1 (Dense)	(None, 128)	100,480
dropout (Dropout)	(None, 128)	0
hidden_2 (Dense)	(None, 64)	8,256
output (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Figure 5.2: Output: dense network summary

5.5 Training

```
early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
                                           restore_best_weights=True)
history = model.fit(x_train, y_train, validation_data=(x_val, y_val),
                    epochs=30, batch_size=128, callbacks=[early_stop])
```



```
✓ 47s
epochs=30, batch_size=128, callbacks=[early_stop], verbose=2)

Epoch 1/30
422/422 - 8s - 19ms/step - accuracy: 0.8822 - loss: 0.3993 - val_accuracy: 0.9410 - val_loss: 0.1942
Epoch 2/30
422/422 - 2s - 6ms/step - accuracy: 0.9489 - loss: 0.1728 - val_accuracy: 0.9578 - val_loss: 0.1410
Epoch 3/30
422/422 - 3s - 7ms/step - accuracy: 0.9612 - loss: 0.1309 - val_accuracy: 0.9652 - val_loss: 0.1212
Epoch 4/30
422/422 - 2s - 5ms/step - accuracy: 0.9682 - loss: 0.1060 - val_accuracy: 0.9663 - val_loss: 0.1134
Epoch 5/30
422/422 - 2s - 5ms/step - accuracy: 0.9724 - loss: 0.0897 - val_accuracy: 0.9710 - val_loss: 0.1015
Epoch 6/30
422/422 - 2s - 6ms/step - accuracy: 0.9754 - loss: 0.0777 - val_accuracy: 0.9723 - val_loss: 0.0934
Epoch 7/30
422/422 - 2s - 6ms/step - accuracy: 0.9779 - loss: 0.0690 - val_accuracy: 0.9747 - val_loss: 0.0954
Epoch 8/30
422/422 - 2s - 5ms/step - accuracy: 0.9803 - loss: 0.0621 - val_accuracy: 0.9757 - val_loss: 0.0933
Epoch 9/30
422/422 - 2s - 5ms/step - accuracy: 0.9817 - loss: 0.0555 - val_accuracy: 0.9767 - val_loss: 0.0884
Epoch 10/30
422/422 - 2s - 5ms/step - accuracy: 0.9837 - loss: 0.0496 - val_accuracy: 0.9757 - val_loss: 0.0895
Epoch 11/30
422/422 - 2s - 6ms/step - accuracy: 0.9850 - loss: 0.0443 - val_accuracy: 0.9777 - val_loss: 0.0941
Epoch 12/30
422/422 - 2s - 5ms/step - accuracy: 0.9859 - loss: 0.0428 - val_accuracy: 0.9775 - val_loss: 0.0848
Epoch 13/30
422/422 - 3s - 7ms/step - accuracy: 0.9873 - loss: 0.0379 - val_accuracy: 0.9762 - val_loss: 0.0975
Epoch 14/30
422/422 - 2s - 6ms/step - accuracy: 0.9876 - loss: 0.0356 - val_accuracy: 0.9773 - val_loss: 0.0892
Epoch 15/30
422/422 - 2s - 5ms/step - accuracy: 0.9881 - loss: 0.0344 - val_accuracy: 0.9755 - val_loss: 0.0974
Epoch 16/30
422/422 - 2s - 6ms/step - accuracy: 0.9881 - loss: 0.0339 - val_accuracy: 0.9797 - val_loss: 0.0927
Epoch 17/30
422/422 - 3s - 7ms/step - accuracy: 0.9906 - loss: 0.0281 - val_accuracy: 0.9777 - val_loss: 0.0922
Epoch 17: early stopping
Restoring model weights from the end of the best epoch: 12.
```

Figure 5.3: Output: training log of the dense network

5.6 Building and Training the CNN

```
cnn = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),
    keras.layers.Conv2D(32, (3, 3), activation="relu"),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(64, (3, 3), activation="relu"),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation="softmax"),
])
cnn.fit(x_train[..., None], y_train, validation_data=(x_val[..., None], y_val),
        epochs=15, batch_size=128, callbacks=[keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=3, restore_best_weights=True)])
```

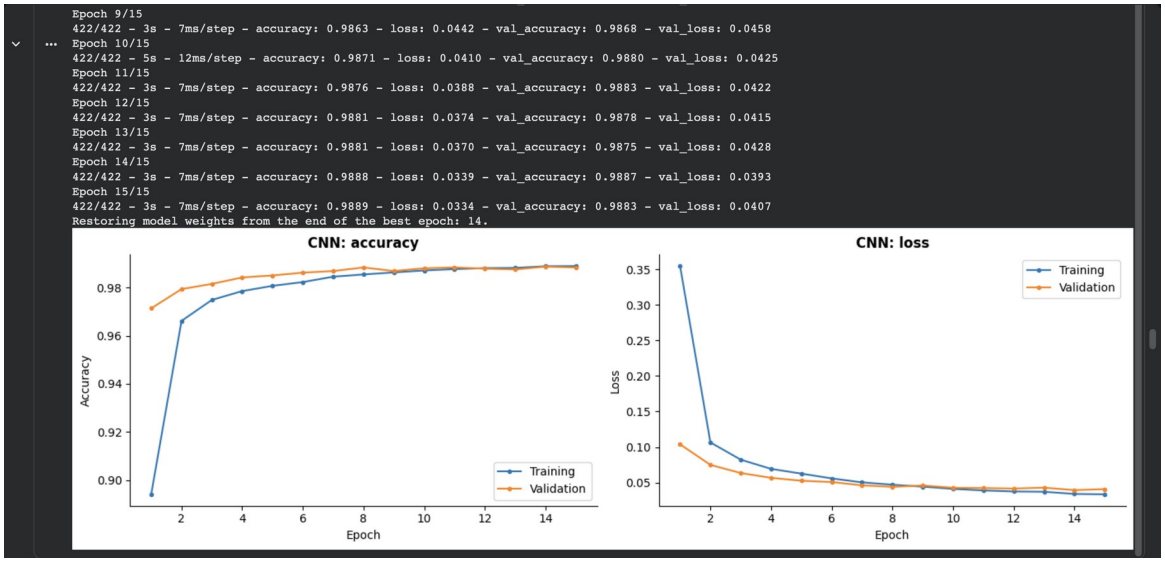


Figure 5.4: Output: training log of the CNN

5.7 Evaluation

```
probs = model.predict(x_test)                    # 10 probabilities per image
pred = np.argmax(probs, axis=1)                     # most likely digit
print(classification_report(y_test, pred, digits=4))
cm = confusion_matrix(y_test, pred)
```

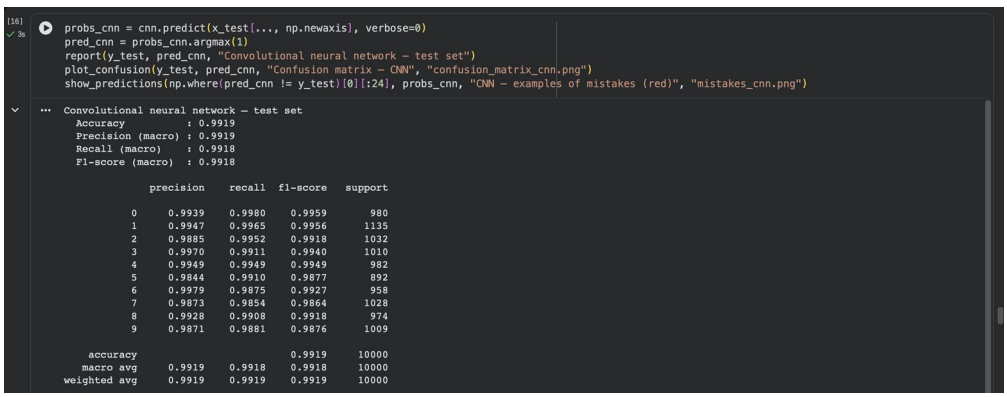


Figure 5.5: Output: test accuracy and classification report

5.8 Prediction System

The function prepare_image (src/preprocess.py) implements the steps of Section 4.5. predict_digit prepares the image, runs the model and returns the digit with its confidence and the two runner-up digits. In Google Colab, an HTML canvas lets the user draw with the mouse or a finger; the drawing is returned to Python as a PNG image through google.colab.output.eval_js. Users can also upload a photo, or run the command-line tool:

```
python -m src.predict --image my_digit.jpg      # CNN by default
python -m src.predict --image a.png b.png --model mlp
```

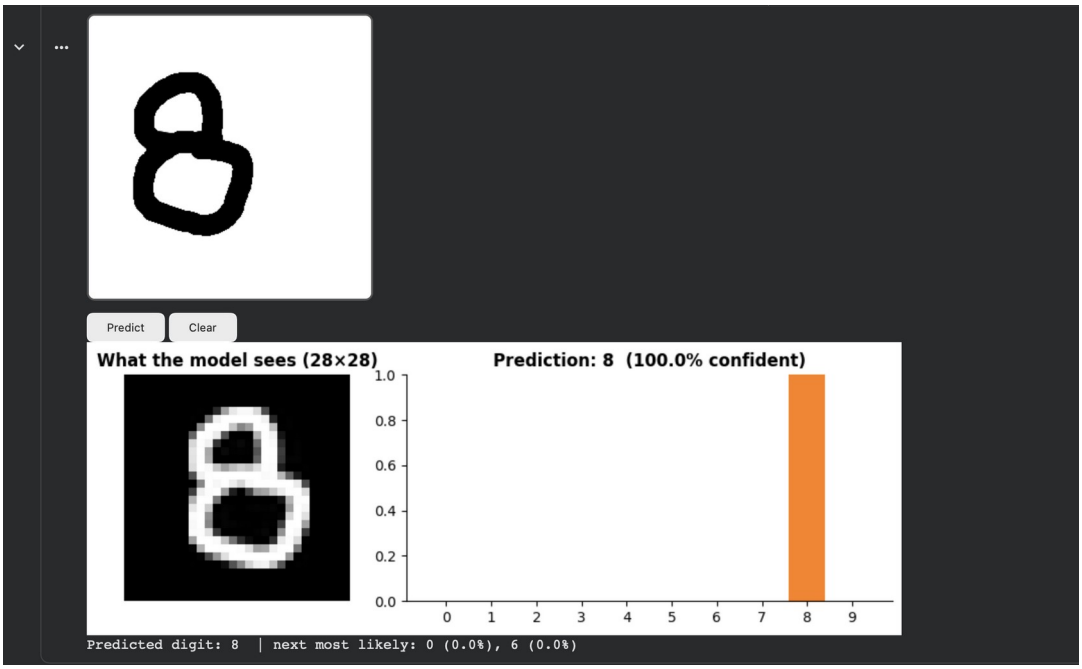


Figure 5.6: Output: the CNN reading an 8 drawn on the Colab canvas

5.9 Browser Demo

To show the model working outside Python, the trained CNN's weights were exported to a JavaScript file (src/export_web.py), and a web page (web_demo/index.html) repeats the same calculations: convolution, ReLU, max-pooling, dense layer and softmax, together with the same preprocessing. The page's predictions match the Python model to within 0.000001 in probability. Users draw a digit and see the prediction, the 28 × 28 image the model receives and the probability of every digit (Figure 5.7).

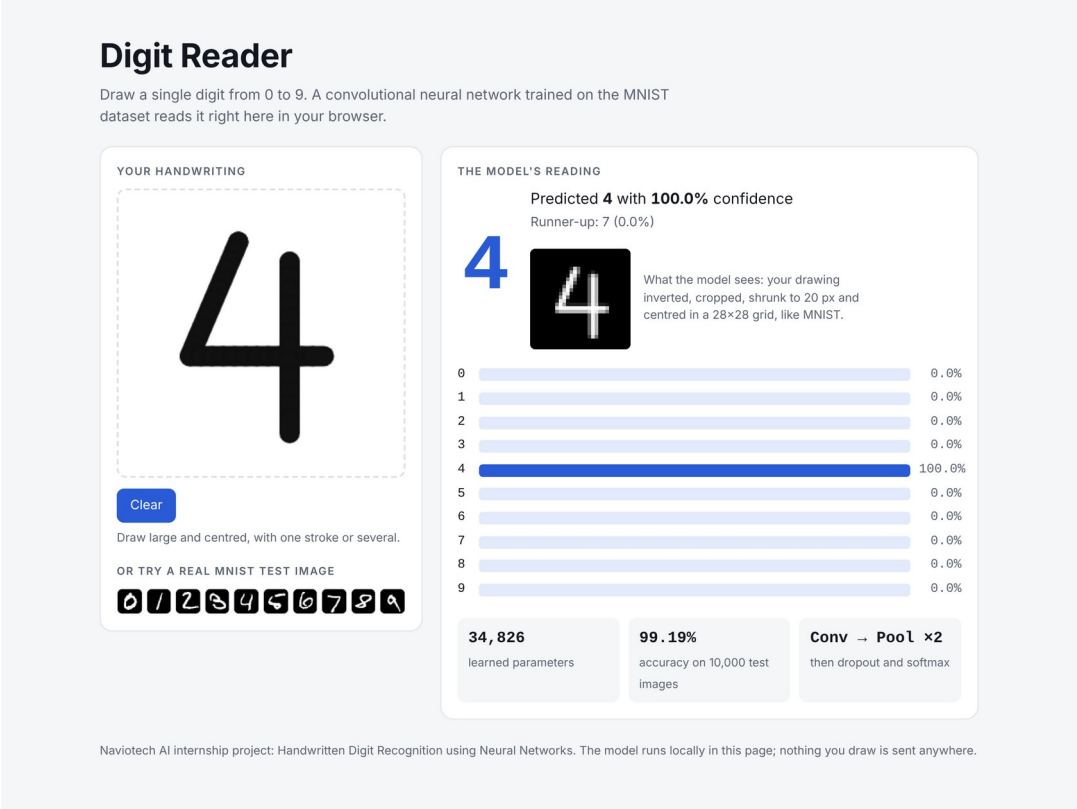


Figure 5.7: Browser demo: the CNN reading a hand-drawn 4

5.10 Saving the Models

Both networks are saved in the native Keras format (models/mnist_mlp.keras and models/mnist_cnn.keras) and reloaded with keras.models.load_model for prediction.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Training Behaviour

Figure 6.1 shows accuracy and loss per epoch. Both networks learn quickly: validation accuracy is already above 95% after two epochs. The dense network's validation loss reached its minimum at epoch 12 and early stopping ended training at epoch 17; at the best epoch its training and validation accuracy were 98.59% and 97.75%. The widening gap between its training and validation loss after about epoch 5 is a sign of mild over-fitting, which early stopping limits. The CNN's best epoch was 14 of 15, with 98.88% training and 98.87% validation accuracy. Its training and validation curves stay together, showing that the CNN generalises better. (Training accuracy is computed with dropout switched on, which is why in the first epochs it is lower than the validation accuracy.)

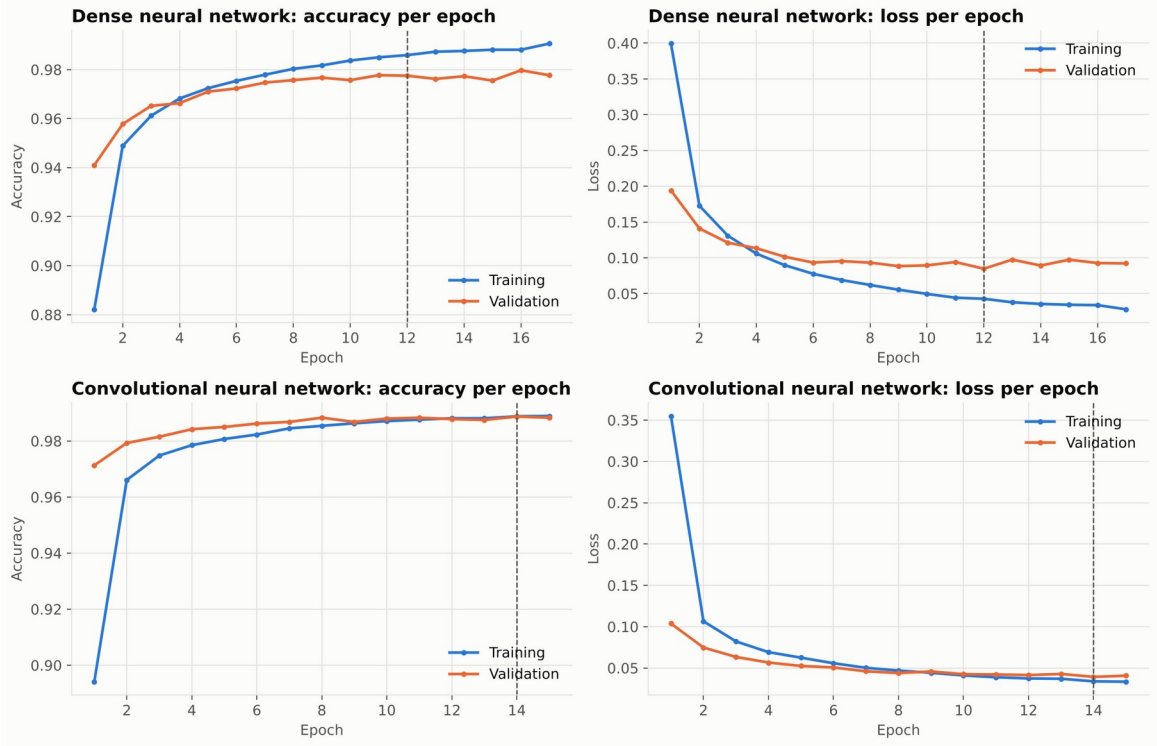


Figure 6.1: Training and validation accuracy and loss for both networks

6.2 Performance on the Test Set

Table 6.1: Test-set performance (10,000 images)

Metric	Dense network	CNN
Accuracy	97.93%	99.19%
Macro precision	97.92%	99.19%
Macro recall	97.91%	99.18%
Macro F1-score	97.91%	99.18%

Metric	Dense network	CNN
Errors	207	81
Test loss	0.0722	0.0235
Parameters	109,386	34,826
Epochs trained (best)	17 (12)	15 (14)

Both networks are well above the 95% target. The CNN makes 81 errors on 10,000 images against 207 for the dense network, a reduction of 61%. With 10,000 test images, a 95% confidence interval for the CNN's accuracy is roughly ±0.2 percentage points, so the gap between the two models is much bigger than what chance could explain.

6.3 Per-Digit Results

Table 6.2: Per-digit recall (accuracy) and F1-score

Digit	Dense recall	Dense F1	CNN recall	CNN F1
0	98.88%	98.73%	99.80%	99.59%
1	98.94%	99.07%	99.65%	99.56%
2	97.67%	98.15%	99.52%	99.18%
3	98.02%	97.30%	99.11%	99.40%
4	97.15%	97.90%	99.49%	99.49%
5	97.42%	97.53%	99.10%	98.77%
6	98.02%	98.02%	98.75%	99.27%
7	98.44%	97.87%	98.54%	98.64%
8	97.74%	97.24%	99.08%	99.18%
9	96.83%	97.31%	98.81%	98.76%

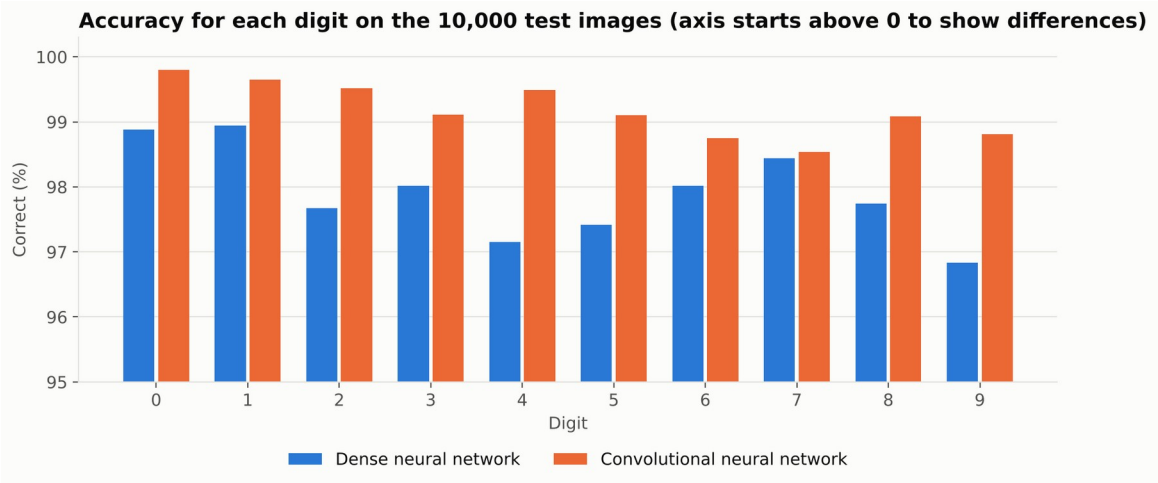


Figure 6.2: Accuracy for each digit

The digits 0 and 1 are the easiest for both models (about 99% for the dense network and above 99.6% for the CNN). For the dense network the hardest digits are 9 (96.8%) and 4 (97.2%). For the CNN the hardest are 7 (98.5%), 6 (98.8%) and 9 (98.8%). The CNN is better than the dense network on every one of the ten digits.

6.4 Confusion Matrices

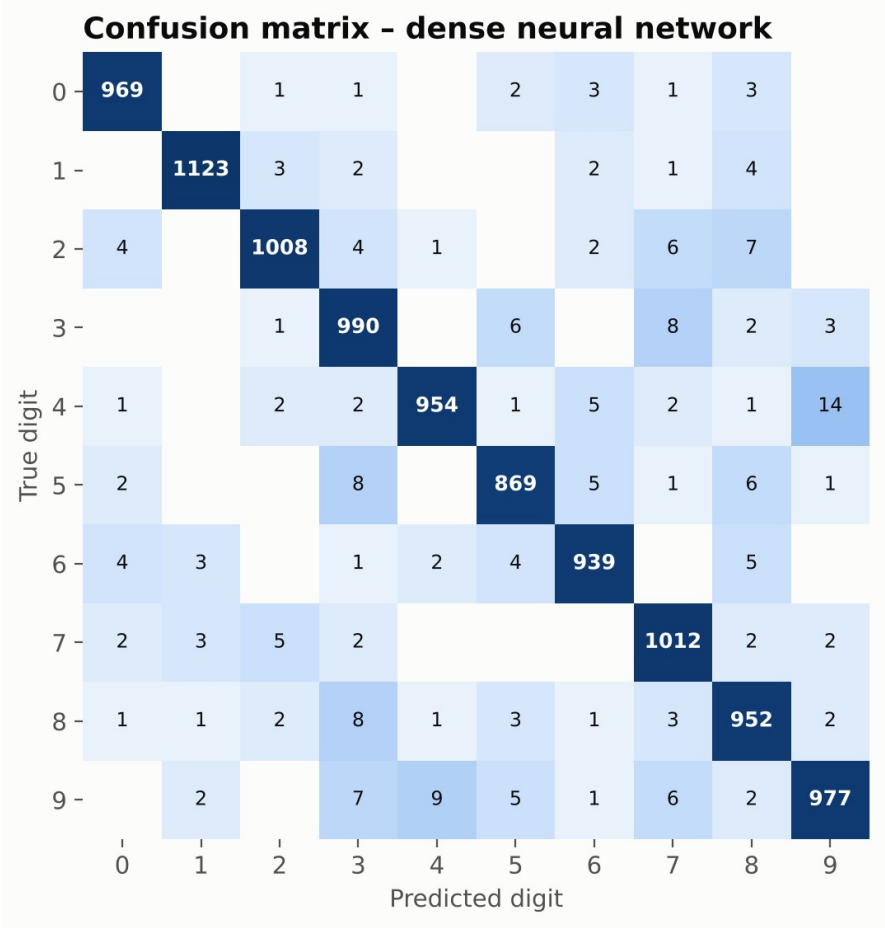


Figure 6.3: Confusion matrix of the dense network

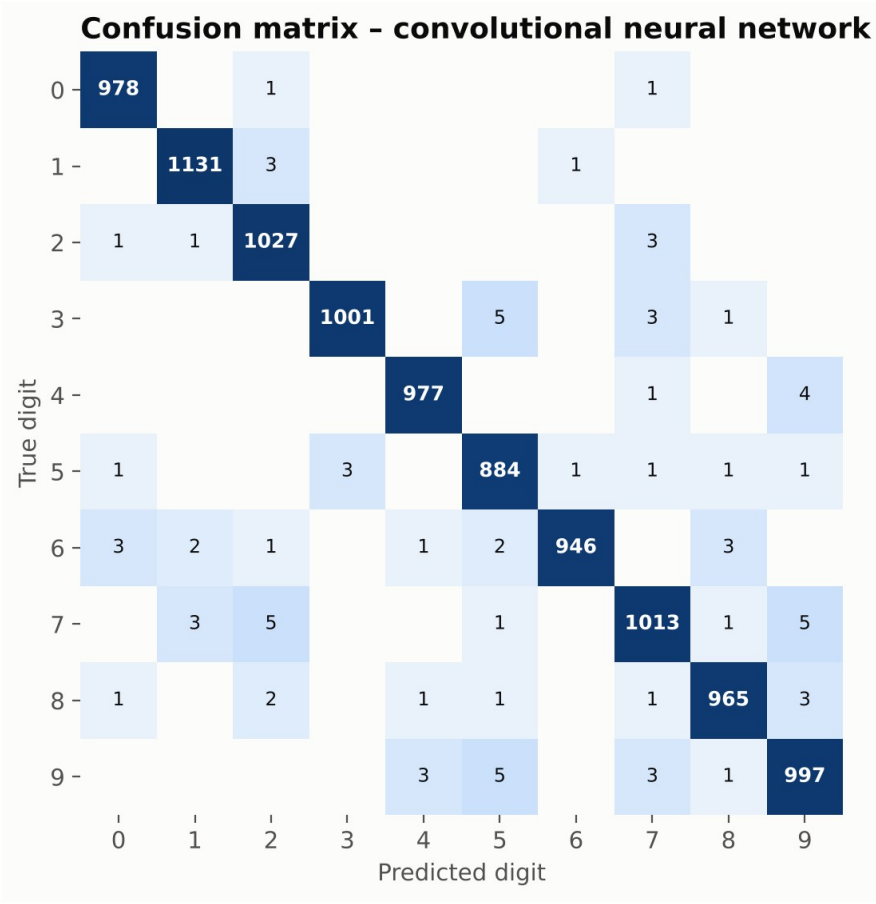


Figure 6.4: Confusion matrix of the CNN

The most frequent confusions of the dense network are 4 → 9 (14 images), 9 → 4 (9) and 8 → 3 (8). For the CNN they are 9 → 5 (5), 7 → 9 (5) and 7 → 2 (5). These pairs look alike: a 4 with a closed top looks like a 9 (and the other way round), an 8 with a weak loop can look like a 3, a 9 with an open loop can look like a 5, and a 7 can be read as a 9 or, if its stem is curved, as a 2.

6.5 Error and Confidence Analysis

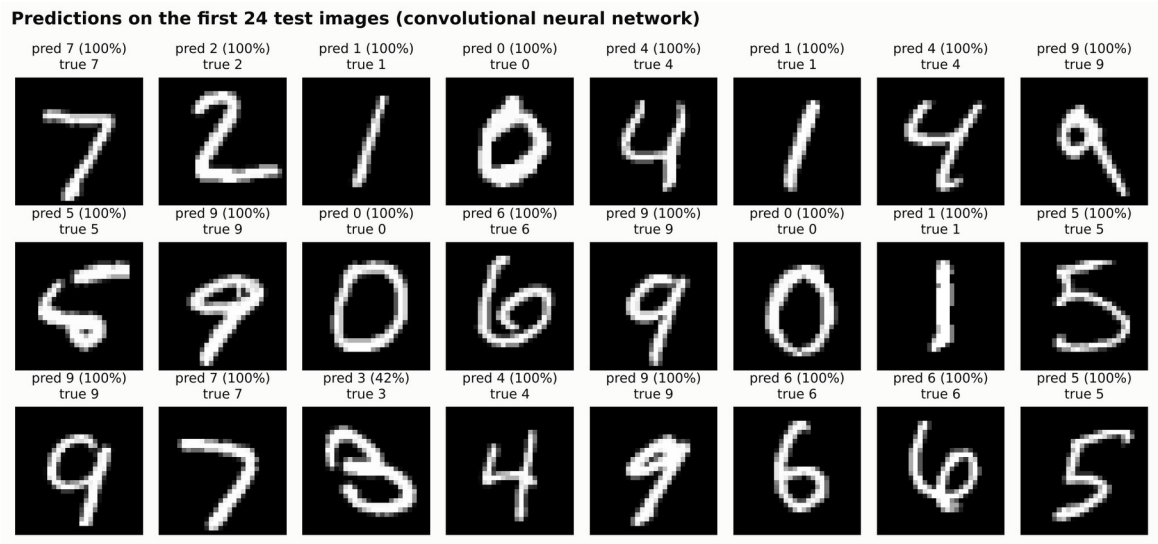


Figure 6.5: CNN predictions on the first 24 test images

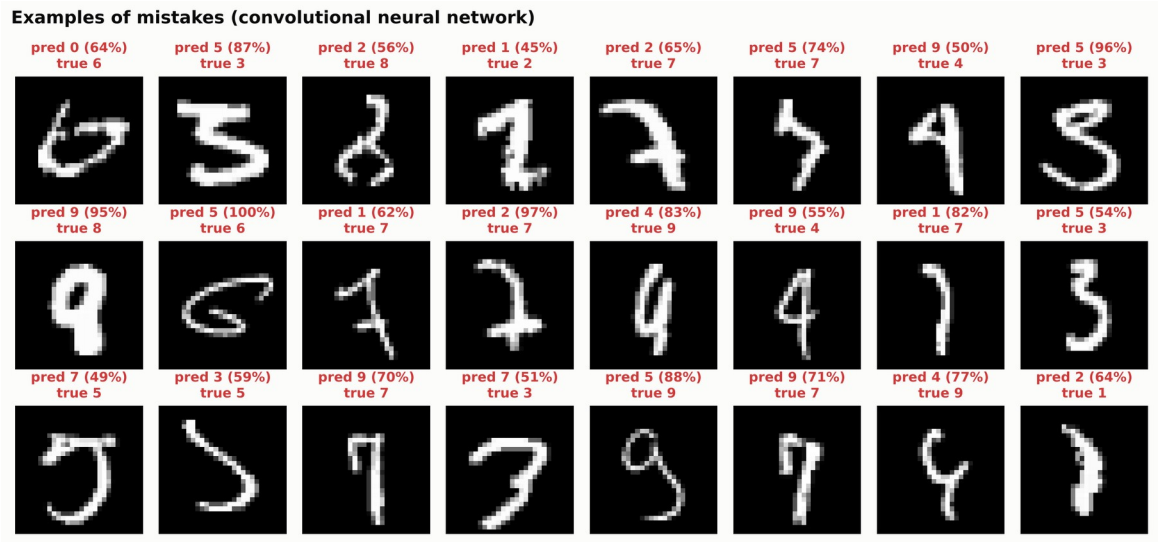


Figure 6.6: Examples of the CNN's mistakes (prediction, confidence and true label)

Among the wrong predictions (Figure 6.6), many of the digits are badly written or could be read either way, and some are hard to read even for a person. The model's confidence also turned out to be useful (Figure 6.7):

- 93.5% of test images receive a confidence of at least 99% from the CNN, and these are correct 99.97% of the time.
- 80% of the CNN's mistakes have a confidence below 90% (median 73%).
- Only 211 images (2.1%) receive a confidence below 90%; sending just these to a human reviewer would catch most errors.

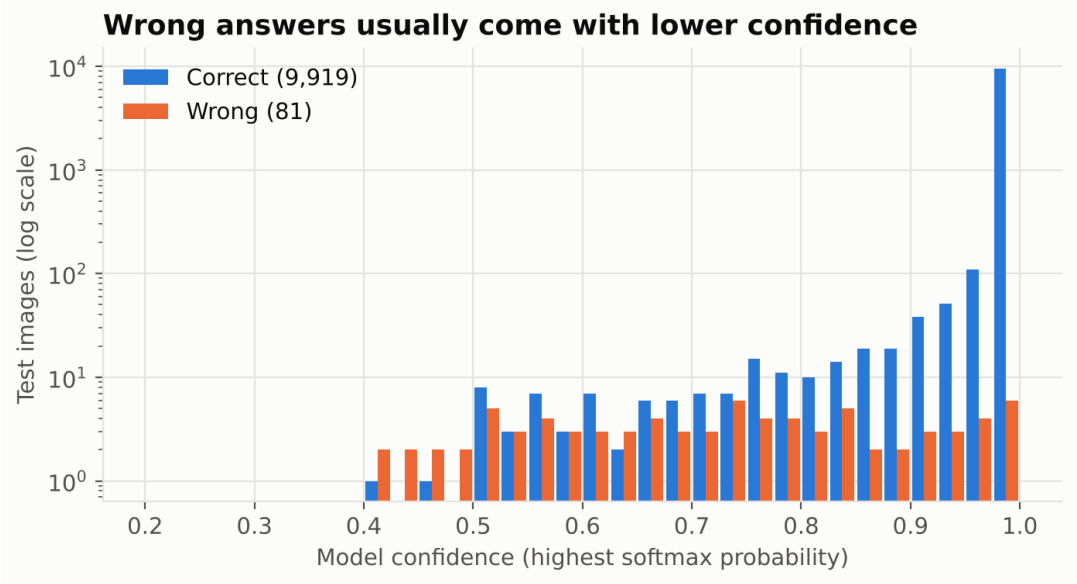


Figure 6.7: Confidence of correct and wrong CNN predictions

6.6 Comparison with Classical Machine Learning

Table 6.3: Test accuracy of all models

Model	Test accuracy	Errors / 10,000
Convolutional neural network	**99.19%**	**81**
Dense neural network	97.93%	207
K-Nearest Neighbours (k=3)	96.99%	301
Random Forest (100 trees)	96.96%	304
Logistic Regression	92.45%	755

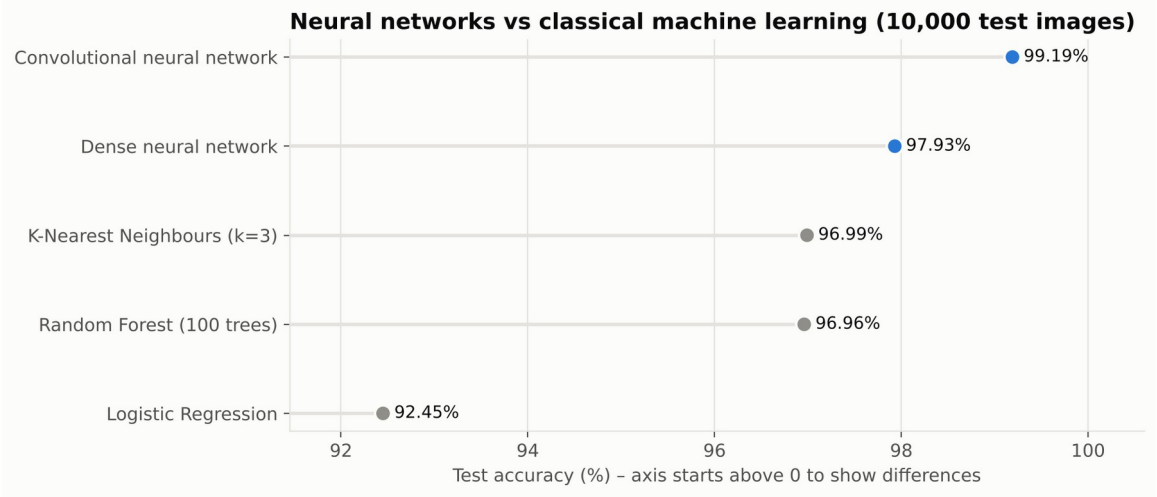


Figure 6.8: Test accuracy of neural networks and classical baselines

Logistic regression can only draw linear boundaries between digits in pixel space and reaches 92.45%. k-nearest neighbours and the random forest do well (about 97%) but are still below the networks. The dense neural network improves on the best classical method

by about one percentage point, and the CNN by two, which shows that learning features works better than comparing raw pixels.

6.7 Comparison with Published Work

Compared with the classic results of LeCun et al. [1] in Table 2.1, the dense network (2.07% error) is much better than the original two- and three-layer networks (4.7% and 2.95%), thanks to ReLU, dropout and the Adam optimiser. The CNN (0.81% error) is slightly better than LeNet-5 (0.95%) while being smaller and simpler to train. The best published results (0.23–0.4%) use data augmentation and large ensembles, which are listed as future work. The classical baselines in this project show errors of 7.55% (logistic regression) and 3.01% (k-nearest neighbours), somewhat lower than the 12.0% and 5.0% reported in [1]; such differences reflect implementation details like the training method, regularisation and the choice of k.

6.8 Recognising Users' Handwriting

The prediction system was tested on two sets of user-style images. The first contains eleven images: six digits drawn with thick strokes on a white canvas, three MNIST test digits restyled as dark ink on white paper, one photo-style image and the "3" supplied in the Naviotech starter material. The CNN recognised all eleven correctly. In Colab, an 8 drawn on the canvas (Figure 5.6) and an uploaded photo of a 6 were also tested; both were read correctly with almost 100% confidence. The second, harder set (src/photo_test.py) contains 30 photo-style images: each of ten sample digits was shrunk to 60–110 pixels and placed at a random position on a 400 × 400 grey, noisy "paper" background, three times each. Table 6.4 compares the MNIST-style preprocessing with the plain resize used in the starter code.

Table 6.4: Accuracy on photo-style user images

Preprocessing	CNN correct
Plain resize to 28 × 28 and invert (starter approach)	4 / 30
MNIST-style preprocessing (Section 4.5)	16 / 30

With a plain resize, a small digit in a large photo becomes a few faint pixels in a corner of the 28 × 28 input, which looks nothing like the centred MNIST digits the model learned from. MNIST-style preprocessing quadrupled the number of correct answers, but it still failed on almost half of these images. The failures show two patterns. Mistakes were more common for the smaller digits (9 of the 13 digits under 85 pixels were wrong, against 5 of the 17 larger ones), and in 8 of the 14 failures the model answered "1". This suggests that background noise was treated as ink, so the crop did not fit the digit properly and the model saw something closer to a thin line. So the system works well on clear drawings and photos, but small, faint digits on noisy paper are still a weakness.

6.9 Discussion

- **Target met.** Both models exceed 95%: the dense network reaches 97.93% and the CNN 99.19%.
- **Architecture matters more than size.** The CNN makes less than half the errors with about three times fewer parameters, because convolution and pooling build in knowledge about images: locality and tolerance to shifts.
- **Mistakes are understandable.** Most of the remaining errors are on digits that look alike or are written unusually, and most are made with low confidence.
- **Deployment needs care.** High test accuracy does not automatically mean the model works on real input. Preparing user images like the training data made a big difference, but small digits on noisy photos are still hard.

6.10 Limitations

- The system recognises one centred digit per image; it cannot segment multi-digit numbers.
- MNIST writers were mostly American adults and students; other handwriting styles (for example, a 1 with a long base or a crossed 7) may be recognised less reliably.
- The model always outputs a digit, even for an image that contains no digit; it does not reject invalid input.
- No data augmentation was used, so robustness to strong rotation or distortion is limited.

CHAPTER 7
CONCLUSION

This project developed a handwritten digit recognition system using neural networks, following the six phases of the Naviotech project guide. The MNIST dataset of 70,000 images was explored, normalised and split into training, validation and test sets so that the final evaluation was unbiased. The main model, a dense neural network with two hidden layers (109,386 parameters), achieved 97.93% accuracy on 10,000 unseen test images, well above the 95% target. An improved convolutional neural network achieved 99.19% with a macro F1-score of 0.9918, with less than half the errors while using about three times fewer parameters.

Both networks outperformed logistic regression, k-nearest neighbours and a random forest, and the results are consistent with the classic literature on MNIST. Error analysis showed that most mistakes involve digits that look alike and are made with low confidence, so the confidence score could be used to flag uncertain cases.

The prediction system lets users draw or upload their own digits, and there is also a command-line tool and a browser demo. Preparing user images like MNIST digits was important: on harder photo-style images it raised the number of correct answers from 4 to 16 out of 30, although very small digits on noisy paper are still a problem.

This project gave me hands-on practice with the whole deep-learning process for images: exploring the data, preprocessing, building dense and convolutional networks, using dropout and early stopping, evaluating the models, studying their mistakes and finally deploying them. The biggest thing I learned is that a model that scores 99% on the test set can still struggle with real photos if the input is not prepared the same way as the training data.

CHAPTER 8

FUTURE SCOPE

- **Data augmentation.** Training on randomly rotated, shifted, scaled and elastically distorted digits [7] would improve robustness and accuracy.
- **Deeper networks.** Additional convolutional layers with batch normalisation, or ensembles of CNNs [8], could reduce the error towards 0.3%.
- **Rejecting unclear input.** A confidence threshold could return "not sure" instead of guessing, routing difficult cases to a human.
- **Multi-digit and character recognition.** Segmenting numbers into digits, or training on EMNIST letters, would extend the system to postcodes, amounts and words.
- **More diverse handwriting.** Fine-tuning on digits written in other styles and scripts, such as Devanagari numerals, would broaden its use.
- **Mobile deployment.** Converting the CNN to TensorFlow Lite would allow offline recognition on smartphones.
- **Explainability.** Visualising learned filters and saliency maps would show which strokes drive each decision.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] Y. LeCun, C. Cortes and C. J. C. Burges, "The MNIST database of handwritten digits." [Online]. Available: http://yann.lecun.com/exdb/mnist/

[3] L. Deng, "The MNIST database of handwritten digit images for machine learning research," IEEE Signal Processing Magazine, vol. 29, no. 6, pp. 141–142, 2012.

[4] R. Plamondon and S. N. Srihari, "On-line and off-line handwriting recognition: A comprehensive survey," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 1, pp. 63–84, 2000.

[5] D. H. Hubel and T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," Journal of Physiology, vol. 160, no. 1, pp. 106–154, 1962.

[6] K. Fukushima, "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position," Biological Cybernetics, vol. 36, pp. 193–202, 1980.

[7] P. Y. Simard, D. Steinkraus and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in Proc. 7th Int. Conf. Document Analysis and Recognition (ICDAR), 2003, pp. 958–963.

[8] D. Cireşan, U. Meier and J. Schmidhuber, "Multi-column deep neural networks for image classification," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3642–3649.

[9] A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems 25, 2012, pp. 1097–1105.

[10] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533–536, 1986.

[11] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in Proc. 27th Int. Conf. Machine Learning (ICML), 2010, pp. 807–814.

[12] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015.

[14] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in Proc. 13th Int. Conf. Artificial Intelligence and Statistics (AISTATS), 2010, pp. 249–256.

[15] L. Prechelt, "Early stopping – but when?" in Neural Networks: Tricks of the Trade, G. B. Orr and K.-R. Müller, Eds. Berlin: Springer, 1998, pp. 55–69.

[16] Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[17] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[18] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[19] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[20] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[21] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[22] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io

APPENDIX A: SOURCE CODE

The complete project, including the Jupyter notebook, training pipeline, browser demo and visualisation code, is available on GitHub at github.com/sangini-agrawal/mnist-digit-classification-nn. The core modules are reproduced below.

A.1 src/model.py

```
"""Phase 3: model development – a dense neural network and a convolutional neural
network."""
from tensorflow import keras

from . import config


def _compile(model):

model.compile(optimizer=keras.optimizers.Adam(learning_rate=config.LEARNING_RATE),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model


def build_mlp() -> keras.Sequential:
    """Dense network: 28×28 image -> Flatten (784) -> 128 ReLU -> Dropout -> 64
ReLU -> 10 Softmax."""
    h1, h2 = config.MLP_UNITS
    return _compile(keras.Sequential([
        keras.layers.Input(shape=(28, 28)),
        keras.layers.Flatten(name="flatten"),
        keras.layers.Dense(h1, activation="relu", name="hidden_1"),
        keras.layers.Dropout(config.MLP_DROPOUT, name="dropout"),
        keras.layers.Dense(h2, activation="relu", name="hidden_2"),
        keras.layers.Dense(10, activation="softmax", name="output"),
    ], name="mnist_dense_nn"))


def build_cnn() -> keras.Sequential:
    """Convolutional network: two Conv-ReLU-MaxPool blocks, then Dropout and a 10-
way Softmax."""
    f1, f2 = config.CNN_FILTERS
    return _compile(keras.Sequential([
        keras.layers.Input(shape=(28, 28, 1)),
        keras.layers.Conv2D(f1, (3, 3), activation="relu", name="conv_1"),
        keras.layers.MaxPooling2D((2, 2), name="pool_1"),
        keras.layers.Conv2D(f2, (3, 3), activation="relu", name="conv_2"),
        keras.layers.MaxPooling2D((2, 2), name="pool_2"),
        keras.layers.Flatten(name="flatten"),
        keras.layers.Dropout(config.CNN_DROPOUT, name="dropout"),
        keras.layers.Dense(10, activation="softmax", name="output"),
    ], name="mnist_cnn"))
```

A.2 src/preprocess.py

```
"""Turn a user's handwritten digit image into an MNIST-style 28×28 input.
```

```
MNIST digits are white on black, fitted into a 20×20 box and centred by their
centre
of mass inside a 28×28 frame. A photo or drawing must go through the same steps,
otherwise the model sees something very different from its training data."""
from __future__ import annotations

import numpy as np
from PIL import Image


def _center_of_mass(a: np.ndarray):
    total = a.sum()
    if total == 0:
        return a.shape[0] / 2, a.shape[1] / 2
    rows, cols = np.indices(a.shape)
    return (rows * a).sum() / total, (cols * a).sum() / total


def prepare_image(source, return_steps: bool = False):
    """source: file path, PIL image or numpy array. Returns a float32 (28, 28)
array in [0, 1]."""
    if isinstance(source, Image.Image):
        img = source
    elif isinstance(source, np.ndarray):
        img = Image.fromarray(source.astype(np.uint8))
    else:
        img = Image.open(source)
    if img.mode in ("RGBA", "LA"):                        # transparent drawings:
paste on white
        bg = Image.new("RGBA", img.size, (255, 255, 255, 255))
        img = Image.alpha_composite(bg, img.convert("RGBA"))
    gray = np.asarray(img.convert("L"), dtype=np.float32)
    steps = {"original": gray.copy()}

    # 1. MNIST is white digit on black: invert if the background is light
    if np.median(gray) > 127:
        gray = 255.0 - gray
    # remove the paper/background level and faint noise (thresholds adapt to the
ink strength)
    gray = np.clip(gray - np.median(gray), 0, None)
    peak = np.percentile(gray, 99.9)
    if peak <= 0:
        raise ValueError("The image looks empty – no digit found.")
    gray[gray < 0.15 * peak] = 0
    steps["inverted"] = gray.copy()

    # 2. Crop to the digit's bounding box (using only clearly inked pixels)
    ys, xs = np.nonzero(gray >= 0.3 * peak)
    if len(ys) == 0:
        raise ValueError("The image looks empty – no digit found.")
    crop = np.clip(gray[ys.min():ys.max() + 1, xs.min():xs.max() + 1] * (255.0 /
peak), 0, 255)
    steps["cropped"] = crop.copy()

    # 3. Resize so the longer side is 20 px, keeping the aspect ratio
    h, w = crop.shape
    scale = 20.0 / max(h, w)
    nh, nw = max(1, round(h * scale)), max(1, round(w * scale))
```

```
    small = np.asarray(Image.fromarray(crop.astype(np.uint8)).resize((nw, nh),
Image.LANCZOS), dtype=np.float32)

    # 4. Paste into a 28×28 frame, centred by centre of mass
    canvas = np.zeros((28, 28), dtype=np.float32)
    top, left = (28 - nh) // 2, (28 - nw) // 2
    canvas[top:top + nh, left:left + nw] = small
    cy, cx = _center_of_mass(canvas)
    dy, dx = int(round(14 - cy)), int(round(14 - cx))
    canvas = np.roll(np.roll(canvas, dy, axis=0), dx, axis=1)

    out = np.clip(canvas / max(canvas.max(), 1e-6), 0, 1).astype(np.float32)
    steps["final"] = out
    return (out, steps) if return_steps else out
```